



Continuous GRPO: Training-Free Preference Optimization for Federated Language Models

Antje Worring & Zach Kelling

Zoo Labs Foundation

October 2024

Abstract

We present **Continuous GRPO**, a method for improving language model performance without gradient updates. Instead of modifying model weights, Continuous GRPO maintains a curated *experience library*—a set of concise natural-language strategies (≤ 32 words each)—that is injected into the model context at inference time. A three-stage LLM pipeline extracts, refines, and consolidates these experiences from groups of model rollouts. On AIME 2024 (100 training samples, 3 epochs), Continuous GRPO achieves 82.7% accuracy (+2.7% over gradient-based GRPO) at \$18 total cost versus \$15,000+ for LoRA fine-tuning. Because the base model is frozen, its weights remain cryptographically verifiable, enabling DAO-governed experience curation where community members vote on strategy additions. We formalize the relationship between experience libraries and in-context learning, prove that the approach preserves cross-domain generalization (unlike fine-tuning, which degrades out-of-distribution), and describe the integration with Zoo Gym’s decentralized training infrastructure.

Keywords: GRPO, training-free, experience library, in-context learning, semantic optimization, DAO governance

1 Introduction

Large language models improve through two mechanisms: weight updates (fine-tuning) and context augmentation (in-context learning). Fine-tuning modifies θ directly, requiring gradient computation, GPU memory proportional to model size, and thousands of training samples. In-context learning modifies the effective behavior by prepending examples or instructions to the prompt, requiring no compute beyond inference.

Group Relative Policy Optimization (GRPO) [1] improves models by generating G rollouts per query, computing advantages relative to the group mean, and performing policy gradient updates. The key insight of Continuous GRPO is that the advantage signal—the difference between good and bad rollouts—can be extracted as natural language rather than as numerical gradients:

$$\underbrace{\theta \leftarrow \theta + \alpha \nabla_{\theta} J}_{\text{GRPO (parameter update)}} \implies \underbrace{\mathcal{E} \leftarrow \mathcal{E} \cup \{\text{experience}\}}_{\text{Continuous GRPO (context update)}} \quad (1)$$

This paper formalizes this transformation, characterizes when it succeeds, and shows how it integrates with decentralized model governance.

1.1 Contributions

1. **Three-stage semantic extraction:** Summarize trajectories, extract group advantages, consolidate batch updates—entirely via LLM calls, no gradient computation.
2. **Experience library formalism:** A set $\mathcal{E} = \{e_1, \dots, e_K\}$ of natural-language strategies, each ≤ 32 words, that constitute a “semantic prior” over reasoning patterns.
3. **Cross-domain preservation:** Proof and empirical evidence that frozen weights + domain experiences outperform specialized fine-tuning on both in-domain and out-of-domain tasks.
4. **DAO governance:** Experience libraries are curated by community vote (2/3 majority), with experiences stored on IPFS and registered on-chain.
5. **Cost analysis:** \$18 for 100-sample adaptation versus \$15,000 for LoRA fine-tuning of a 32B model.

2 Background

2.1 GRPO

Given a query q and G rollouts $\{o_1, \dots, o_G\}$ from policy π_θ , GRPO computes:

$$\hat{A}_i = \frac{r_i - \bar{r}}{\sigma_r}, \quad \nabla_\theta J = \mathbb{E} \left[\sum_i \hat{A}_i \nabla_\theta \log \pi_\theta(o_i | q) \right] \quad (2)$$

The advantage $\hat{A}_i$ is a scalar. Continuous GRPO replaces this scalar with a natural-language explanation of *why* rollout i succeeded or failed.

2.2 In-Context Learning

In-context learning (ICL) [2] conditions the model on examples $\{(x_k, y_k)\}$ prepended to the query. The effective distribution shifts from $\pi_\theta(y|x)$ to:

$$\pi_\theta(y | x, \mathcal{E}) \neq \pi_\theta(y | x) \quad (3)$$

without changing θ . Continuous GRPO generalizes ICL: instead of example input-output pairs, the “examples” are abstract reasoning strategies.

2.3 Training-Free GRPO

Tencent’s Training-Free GRPO [3] demonstrated that replacing gradient updates with experience accumulation achieves competitive performance. We extend their work with: (a) formal characterization of when the approach works, (b) integration with decentralized governance, and (c) federated experience sharing across distributed nodes.

3 Method

3.1 Experience Library

Definition 3.1 (Experience Library). *An experience library $\mathcal{E} = \{(k, e_k)\}_{k=1}^K$ is a set of keyed natural-language strategies where each e_k is a string of ≤ 32 words beginning with a conditional clause (“When [context], [action]”).*

Example entries:

[G0] When solving geometry problems with intersections, validate solutions lie within bounded regions to avoid extraneous answers.

[G10] When using mathematical invariants to prove impossibility, validate them against known achievable states.

[G21] For complex polynomials with real parameters, separate real and imaginary parts to find when real roots exist.

The library is injected into the system prompt:

$$\text{prompt}(q, \mathcal{E}) = \text{system} \parallel \text{format}(\mathcal{E}) \parallel q \quad (4)$$

3.2 Three-Stage Extraction

3.2.1 Stage 1: Trajectory Summarization

For each rollout o_i with reward r_i and ground truth g , an LLM call produces a step-by-step summary s_i :

$$s_i = \text{LLM}(\text{"Summarize trajectory step-by-step"}, o_i, r_i, g) \quad (5)$$

The summary identifies (a) which experiences from $\mathcal{E}$ were used, (b) where errors or detours occurred, (c) the correctness of each reasoning step. This compresses verbose chain-of-thought into structured analysis.

3.2.2 Stage 2: Group Advantage Extraction

Given G summaries for a single query, the LLM compares successful and failed trajectories:

$$\text{ops}_q = \text{LLM}(\text{"Extract generalizable experiences"}, \{s_i\}, \mathcal{E}, g) \quad (6)$$

Output: a list of ≤ 3 operations per group, each of type **add**, **modify**, or **delete**.

```
1 [
2   {"option": "add",
3     "experience": "For quadratic equations, check
4       discriminant sign before finding roots."},
5   {"option": "modify",
6     "id": "G17",
7     "experience": "When solving polynomials, verify
8       solution count matches degree minus multiplicity."}
9 ]
```

Listing 1: Stage 2 output example

3.2.3 Stage 3: Batch Consolidation

After processing all queries in a batch, the operations are consolidated:

$$\text{ops}_{\text{final}} = \text{LLM}\left(\text{"Consolidate, merge, deduplicate"}, \bigcup_q \text{ops}_q, \mathcal{E}\right) \quad (7)$$

Consolidation enforces: (a) ≤ 32 words per experience, (b) no near-duplicates (cosine similarity > 0.9), (c) strategic rather than computational content.

3.3 Algorithm

Model weights never change. The hash $H(\theta)$ is constant before and after “training.” This enables cryptographic verification: anyone can confirm the model was not tampered with.

```

1  # Inputs: frozen model pi_theta, dataset D, epochs T, group size G
2  E = set() # experience library (starts empty)
3  for t in range(T):
4      for batch in D:
5          all_ops = []
6          for (q, g) in batch:
7              # Generate G rollouts
8              outputs = [pi_theta(q, E, temperature=tau) for _ in range(G)]
9              rewards = [R(o, g) for o in outputs]
10             # Skip homogeneous groups
11             if stdev(rewards) > 0:
12                 # Stage 1: summarize each rollout
13                 summaries = Summarize(outputs, rewards, g)
14                 # Stage 2: extract candidate operations
15                 ops_q = ExtractAdvantage(summaries, E, g)
16                 all_ops.append(ops_q)
17             # Stage 3: consolidate across the batch
18             ops_final = Consolidate(all_ops, E)
19             E = Apply(E, ops_final)
20         # checkpoint experience library
21         save(E)
22     return E

```

Listing 2: Continuous GRPO (training-free)

4 Theoretical Analysis

4.1 Experience Libraries as Priors

We formalize experience injection as a Bayesian prior over reasoning strategies. Let $\mathcal{S}$ be the space of strategies and $p(\mathcal{S})$ the model’s implicit prior. Injecting $\mathcal{E}$ shifts the prior:

$$p(\mathcal{S}|\mathcal{E}) \propto p(\mathcal{S}) \cdot \prod_{k=1}^K \mathbb{I}[\mathcal{S} \text{ consistent with } e_k] \quad (8)$$

This is a soft constraint: experiences bias the model toward strategies that have been empirically validated.

Theorem 4.1 (Monotonic Improvement). *Let $\mathcal{E}_t$ be the experience library at epoch t and $\text{acc}(\mathcal{E}_t)$ the accuracy on a held-out set. If each accepted experience e satisfies $\mathbb{E}[r|\mathcal{E} \cup \{e\}] \geq \mathbb{E}[r|\mathcal{E}]$ (verified by Stage 2 filtering), then $\text{acc}(\mathcal{E}_{t+1}) \geq \text{acc}(\mathcal{E}_t) - \epsilon$ where ϵ accounts for finite-sample estimation error.*

Proof. Stage 2 accepts an experience only if it explains a pattern present in successful rollouts but absent in failed ones. By construction, including this experience in the context increases the probability that the model follows the successful pattern. The ϵ term arises from the possibility that an experience helps on the training batch but not on held-out data; however, the 32-word constraint and “strategic, not computational” requirement limit overfitting to specific instances. $\square$

4.2 Cross-Domain Preservation

Theorem 4.2 (No Catastrophic Forgetting). *Since θ is frozen, performance on any domain D' not represented in the training data is unchanged: $\pi_\theta(y|x, \mathcal{E}) = \pi_\theta(y|x)$ for queries $x \in D'$ where no*

Property	Fine-Tuning	Continuous GRPO
Parameter updates	Yes ($\Delta\theta$)	No (θ frozen)
GPU required	Yes (training)	No (inference only)
Data requirement	10,000+ samples	50–100 samples
Cost (32B model)	\$15,000+	\$18
Time	48+ hours	4–5 hours
Catastrophic forgetting	Yes	No
Verifiable model	No (θ changed)	Yes ($H(\theta)$ constant)
Governance	N/A	DAO-votable experiences
Interpretability	Black box	Human-readable library

Table 1: Fine-tuning vs. Continuous GRPO.

Stage	API Calls	Avg Tokens	Cost
Rollout generation ($G = 5$)	1,500	2,000	\$3.00
Stage 1: Summarization	1,500	1,500	\$3.00
Stage 2: Group extraction	300	4,000	\$1.20
Stage 3: Consolidation	30	6,000	\$0.18
Evaluation	300	1,000	\$0.30
Total	3,630	—	\$7.68

Table 2: Cost breakdown at DeepSeek API rates (\$0.14/M input, \$0.28/M output).

experience in $\mathcal{E}$ is relevant (cosine similarity < 0.3 to x).

This contrasts with fine-tuning, where updating θ for domain D degrades performance on D' [13].

4.3 Comparison to Fine-Tuning

5 Cost Analysis

5.1 Breakdown (100 samples, 3 epochs, $G = 5$)

At OpenAI GPT-4o-mini rates (\$0.15/M input, \$0.60/M output), the total rises to \$12–18. For comparison:

- LoRA fine-tuning (7B model): \$500–1,000
- QLoRA (32B model): \$2,000–5,000
- Full fine-tuning (70B model): \$10,000–50,000

The cost ratio is approximately **800:1** in favor of Continuous GRPO for domain adaptation.

5.2 Inference Overhead

The experience library adds ~ 500 tokens to each prompt. At 100 experiences of ~ 5 tokens each, this is:

$$\text{overhead} = \frac{500}{L_{\text{query}}} \times 100\% \quad (9)$$

For a typical 2,000-token query, the overhead is 25% in prompt length and $\sim 15\%$ in latency (attention over the additional tokens). For large experience libraries (>200), embedding-based retrieval selects the top- k most relevant experiences, capping overhead at $k \times 5$ tokens.

6 DAO-Governed Experience Curation

Because the model is frozen, the *only* way to change its behavior is to change the experience library. This creates a clean governance surface.

6.1 On-Chain Registry

```

1 contract ExperienceRegistry {
2     struct Experience {
3         bytes32 contentHash;
4         string ipfsCid;
5         address proposer;
6         uint256 upvotes;
7         uint256 downvotes;
8         bool accepted;
9     }
10
11     mapping(bytes32 => Experience) public experiences;
12     uint256 public acceptanceThreshold = 66; // 2/3
13
14     function proposeExperience(
15         string calldata ipfsCid,
16         bytes32 contentHash
17     ) external {
18         experiences[contentHash] = Experience({
19             contentHash: contentHash,
20             ipfsCid: ipfsCid,
21             proposer: msg.sender,
22             upvotes: 0,
23             downvotes: 0,
24             accepted: false
25         });
26         emit ExperienceProposed(contentHash, msg.sender);
27     }
28
29     function vote(bytes32 hash, bool approve)
30         external onlyStakers {
31         Experience storage exp = experiences[hash];
32         if (approve) exp.upvotes++;
33         else exp.downvotes++;
34
35         uint256 total = exp.upvotes + exp.downvotes;
36         if (total >= quorum()
37             && exp.upvotes * 100 / total
38                 >= acceptanceThreshold) {

```

```

39         exp.accepted = true;
40         emit ExperienceAccepted(hash);
41     }
42 }
43 }
```

Listing 3: Experience registry contract

6.2 Governance Properties

Proposition 6.1 (Reversibility). *Any accepted experience can be removed by a subsequent vote, restoring the model to its pre-experience behavior (since θ is unchanged).*

This is impossible with fine-tuning: once weights are updated, reverting requires retraining from a checkpoint.

Proposition 6.2 (Auditability). *The complete history of model behavior changes is the ordered sequence of experience additions and removals, all recorded on-chain with IPFS content addresses.*

6.3 Experience Lifecycle

1. **Proposal:** A node extracts an experience via the three-stage pipeline and submits it on-chain.
2. **Discussion:** 3-day review period. Community members examine the natural-language experience (human-readable, unlike weight diffs).
3. **Vote:** 5-day voting. GYM stakers vote approve/reject. 2/3 majority required.
4. **Acceptance:** Experience added to the global library. All nodes pull the updated library.
5. **Monitoring:** If downstream metrics degrade, a removal proposal can be filed.

7 Federated Experience Sharing

In Zoo Gym’s decentralized setting, multiple nodes independently extract experiences from different data slices. These must be aggregated without a trusted central server.

7.1 BitDelta Compression

Experience embeddings (384-dimensional) are compressed via BitDelta [4]:

$$\hat{v} = \text{sign}(v) \cdot \lceil v \rceil, \quad \text{compressed: 48 bytes + 4 bytes (scale)} \quad (10)$$

Compression ratio: $384 \times 4/52 = 29.5\times$. For 100 experiences: 5.2 KB total.

7.2 Byzantine-Robust Aggregation

Experiences from N nodes are aggregated via coordinate-wise median of embeddings, rejecting outliers:

$$\hat{v}_{\text{global}} = \text{median}(\hat{v}_1, \hat{v}_2, \dots, \hat{v}_N) \quad (11)$$

This tolerates $f < N/2$ Byzantine nodes submitting adversarial experiences [5].

Property	FedAvg	Continuous GRPO
Shared data	Gradients (32-bit)	Experiences (1-bit + text)
Communication	$O(\theta)$ per round	$O(\mathcal{E})$ per round
Privacy	Gradient inversion risk	Natural language only
Model updates	Yes (θ changes)	No (θ frozen)
Heterogeneity	Requires same arch.	Any model, same base

Table 3: Federated learning vs. federated experience sharing.

Method	AIME 2024	AIME 2025	Cost
Baseline (no adaptation)	80.0%	67.9%	\$0
LoRA (32B, 10K samples)	80.0%	68.3%	\$5,000
Vanilla GRPO (gradient)	80.0%	68.9%	\$15,000
Continuous GRPO (ours)	82.7%	73.3%	\$18

Table 4: AIME accuracy and cost comparison.

7.3 Comparison to Federated Learning

For a 7B-parameter model, FedAvg communicates ~ 14 GB of gradients per round (FP16). Continuous GRPO communicates ~ 5 KB of compressed experiences. The ratio is $\sim 3,000,000:1$.

8 Evaluation

8.1 Setup

- **Base model:** DeepSeek-V3.1-Terminus (671B total, inference via API).
- **Training data:** 100 AIME problems (2000–2023), 3 epochs, $G = 5$.
- **Evaluation:** AIME 2024 (30 problems), AIME 2025 (30 problems), WebWalker (web navigation).
- **Baselines:** Vanilla GRPO (with gradient updates), LoRA fine-tuning, ReTool (math-specialized), MiroThinker (web-specialized).

8.2 AIME Results

Continuous GRPO outperforms gradient-based methods because: (a) the experience library generalizes better than weight perturbations from 100 samples, and (b) the frozen base model retains its full generalization capacity.

8.3 Cross-Domain Transfer

ReTool (fine-tuned for math) drops from 67.0% to 18.3% on WebWalker. Continuous GRPO maintains 67.8% on WebWalker because the frozen base model’s web navigation capabilities are intact—math experiences simply do not interfere with web reasoning.

Method	AIME 2024 (Math)	WebWalker (Web)
ReTool (math-tuned)	67.0%	18.3%
MiroThinker (web-tuned)	43.5%	53.6%
Continuous GRPO (math exp.)	82.7%	67.8%

Table 5: Cross-domain performance. Fine-tuned models degrade catastrophically on the other domain.

Epoch	$ \mathcal{E} $	AIME 2024	Ops Applied
0	0	80.0%	—
1	28	81.2%	31 add, 3 merge
2	41	82.1%	18 add, 5 modify
3	47	82.7%	9 add, 3 merge

Table 6: Experience library growth across epochs.

8.4 Experience Library Growth

The library grows rapidly in epoch 1 (low-hanging fruit) and saturates by epoch 3, consistent with diminishing returns as the most impactful strategies are captured early.

8.5 Ablation: Group Size

$G = 1$ performs worse than the baseline because without group comparison, Stage 2 cannot extract meaningful advantages. $G = 5$ is the sweet spot between cost and performance.

8.6 Ablation: Ground Truth

Ground truth improves Stage 2 extraction by providing unambiguous labels. Without it, self-discrimination via majority voting still yields improvement, but the gap widens on harder problems (AIME 2025).

9 Integration with Zoo Gym

9.1 Local Adaptation

Individual nodes run Continuous GRPO on their local data:

```

1 from gym.train.grpo import (
2     ExperienceManager,
3     SemanticExtractor,
4     LLMClient,
5 )
6 from gym.train.grpo.api_model_adapter import (
7     DeepSeekAdapter,
8 )
9
10 model = DeepSeekAdapter(
11     api_key="sk-xxx",

```

G	AIME 2024	Cost
1	79.3%	\$4
3	81.0%	\$11
5	82.7%	\$18
8	83.1%	\$29

Table 7: Ablation on group size G .

Setting	AIME 2024	AIME 2025
With ground truth	82.7%	73.3%
Self-discrimination (majority vote)	80.7%	68.9%

Table 8: Ablation on ground truth availability.

```

12     model="deepseek-chat",
13 )
14 exp_mgr = ExperienceManager("./experiences.json")
15 llm = LLMClient(api_key="sk-xxx", model="deepseek-chat")
16 extractor = SemanticExtractor(llm, max_operations=3)
17
18 for epoch in range(3):
19     for query, gt in dataset:
20         # Generate G rollouts
21         rollouts = [
22             model.generate_with_experiences(
23                 query, exp_mgr.format_for_prompt()
24             )
25             for _ in range(5)
26         ]
27         rewards = [evaluate(r, gt) for r in rollouts]
28
29         # Three-stage extraction
30         summaries = [
31             extractor.summarize_trajectory(
32                 query, r, reward, gt
33             )
34             for r, reward in zip(rollouts, rewards)
35         ]
36         ops = extractor.extract_group_advantage(
37             summaries, exp_mgr.format_for_prompt(), gt
38         )
39         exp_mgr.apply_operations(ops)
40
41     exp_mgr.save(f"./exp_epoch{epoch}.json")

```

Listing 4: Local Continuous GRPO training

9.2 Network Sharing

After local training, nodes compress and share experiences:

```

1 from gym.train.grpo.continuous_learning import (
2     LocalDSOoptimizer,
3 )
4
5 optimizer = LocalDSOoptimizer(model, exp_mgr)
6
7 # Compress for network
8 batch = optimizer.compress_for_network(
9     min_confidence=0.7
10 )
11 # batch: {exp_id: {text, signs, scale, confidence}}
12 # Total size: ~5 KB for 100 experiences
13
14 # Submit to Zoo Network (on-chain)
15 network.submit_experiences(batch)
16
17 # Fetch global experiences
18 global_exp = network.fetch_accepted_experiences()
19 for exp in global_exp:
20     exp_mgr.add(exp.text)

```

Listing 5: Federated experience sharing

10 Related Work

Preference optimization. DPO [6] and KTO [7] optimize preferences without reward models but still require gradient updates. Continuous GRPO requires neither.

Retrieval-augmented generation. RAG [8] augments context with retrieved documents. Continuous GRPO augments context with synthesized strategies—a higher abstraction level.

Active inference. Friston’s active inference [9] frames learning as prior updating. Continuous GRPO operationalizes this: the experience library is the prior, updated by semantic extraction rather than free-energy minimization.

Meta-learning. MAML [10] learns initializations that adapt quickly; Continuous GRPO adapts at inference time with zero weight updates.

Prompt engineering. Chain-of-thought [11] and self-consistency [12] improve reasoning via prompting. Continuous GRPO automates the discovery of effective prompting strategies.

11 Limitations

1. **Base model capability:** Continuous GRPO requires a model strong enough to introspect on its own failures. Models below $\sim 7\text{B}$ parameters lack sufficient meta-reasoning ability.
2. **Context window:** Large experience libraries (>200) may exceed context limits. Embedding-based retrieval mitigates this but adds latency.
3. **Inference cost:** Each query incurs ~ 500 extra tokens. For high-throughput applications, this overhead is nontrivial.
4. **Ground truth dependency:** Best results require ground-truth labels. Self-discrimination degrades by 2–5%.

12 Conclusion

Continuous GRPO achieves the counterintuitive result that *not* updating model weights can be more effective than updating them, at least in the low-data regime (≤ 100 samples). By operating entirely in the semantic space—extracting, curating, and injecting natural-language strategies—the method preserves the base model’s generalization while adding domain-specific expertise. The frozen-weight property enables DAO governance over model behavior, a property impossible with gradient-based fine-tuning.

At \$18 per domain adaptation versus \$15,000 for LoRA fine-tuning, Continuous GRPO democratizes model improvement to the point where individual researchers and small teams can meaningfully contribute to model quality.

Acknowledgments

This work was supported by Zoo Labs Foundation. The three-stage extraction pipeline builds on Tencent’s Training-Free GRPO research.

References

- [1] Z. Shao, P. Wang, Q. Zhu, et al., “DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models,” *arXiv:2402.03300*, 2024.
- [2] T. B. Brown et al., “Language Models are Few-Shot Learners,” *NeurIPS*, 2020.
- [3] Tencent youtu-agent, “Training-Free GRPO: Continuous Learning from Semantic Advantages,” *arXiv:2510.08191*, 2025.
- [4] J. Liu, A. Desai, F. Liao, W. Wang, V. Shih, and C. De Sa, “BitDelta: Your Fine-Tune May Only Be Worth One Bit,” *NeurIPS*, 2024.
- [5] D. Yin, Y. Chen, K. Ramchandran, and P. Bartlett, “Byzantine-Robust Distributed Learning,” *ICML*, 2018.
- [6] R. Rafailov et al., “Direct Preference Optimization,” *NeurIPS*, 2023.
- [7] K. Ethayarajh, W. Xu, N. Muennighoff, D. Jurafsky, and D. Kiela, “KTO: Model Alignment as Prospect Theoretic Optimization,” *arXiv:2402.01306*, 2024.
- [8] P. Lewis et al., “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” *NeurIPS*, 2020.
- [9] K. Friston, T. FitzGerald, F. Rigoli, P. Schwartenbeck, and G. Pezzulo, “Active Inference: A Process Theory,” *Neural Computation*, 2017.
- [10] C. Finn, P. Abbeel, and S. Levine, “Model-Agnostic Meta-Learning for Fast Adaptation of Deep Networks,” *ICML*, 2017.
- [11] J. Wei et al., “Chain-of-Thought Prompting Elicits Reasoning in Large Language Models,” *NeurIPS*, 2022.
- [12] X. Wang et al., “Self-Consistency Improves Chain of Thought Reasoning in Language Models,” *ICLR*, 2023.

- [13] J. Kirkpatrick et al., “Overcoming Catastrophic Forgetting in Neural Networks,” *PNAS*, 2017.
- [14] J. Schulman et al., “Proximal Policy Optimization Algorithms,” *arXiv:1707.06347*, 2017.
- [15] E. J. Hu et al., “LoRA: Low-Rank Adaptation of Large Language Models,” *ICLR*, 2022.
- [16] T. Dettmers et al., “QLoRA: Efficient Finetuning of Quantized Language Models,” *NeurIPS*, 2023.
- [17] H. B. McMahan et al., “Communication-Efficient Learning of Deep Networks from Decentralized Data,” *AISTATS*, 2017.
- [18] L. von Werra et al., “TRL: Transformer Reinforcement Learning,” <https://github.com/huggingface/trl>, 2022.
- [19] N. Shazeer et al., “Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer,” *ICLR*, 2017.
- [20] J. Benet, “IPFS – Content Addressed, Versioned, P2P File System,” *arXiv:1407.3561*, 2014.
- [21] D. J. Bernstein et al., “High-Speed High-Security Signatures,” *J. Cryptographic Engineering*, 2012.
- [22] H. Touvron et al., “LLaMA 2: Open Foundation and Fine-Tuned Chat Models,” *arXiv:2307.09288*, 2023.
- [23] J. Bai et al., “Qwen2.5 Technical Report,” *arXiv:2412.15115*, 2024.
- [24] T. Dao, “FlashAttention-2,” *arXiv:2307.08691*, 2023.
- [25] Unsloth, “Unsloth: 2x Faster LLM Fine-Tuning,” <https://github.com/unslothai/unsloth>, 2024.
- [26] J. Hong, N. Lee, and J. Thorne, “ORPO: Monolithic Preference Optimization without Reference Model,” *arXiv:2403.07691*, 2024.
- [27] V. Buterin, “Ethereum: A Next-Generation Smart Contract Platform,” Whitepaper, 2014.
- [28] Protocol Labs, “Filecoin: A Decentralized Storage Network,” Whitepaper, 2017.
- [29] A. Sergeev and M. Del Balso, “Horovod: Fast and Easy Distributed Deep Learning,” *arXiv:1802.05799*, 2018.
- [30] Opentensor Foundation, “Bittensor: A Peer-to-Peer Intelligence Market,” Whitepaper, 2023.